

Zen Class Program – Case Study Documentation

Step 1: Opening MongoDB Compass

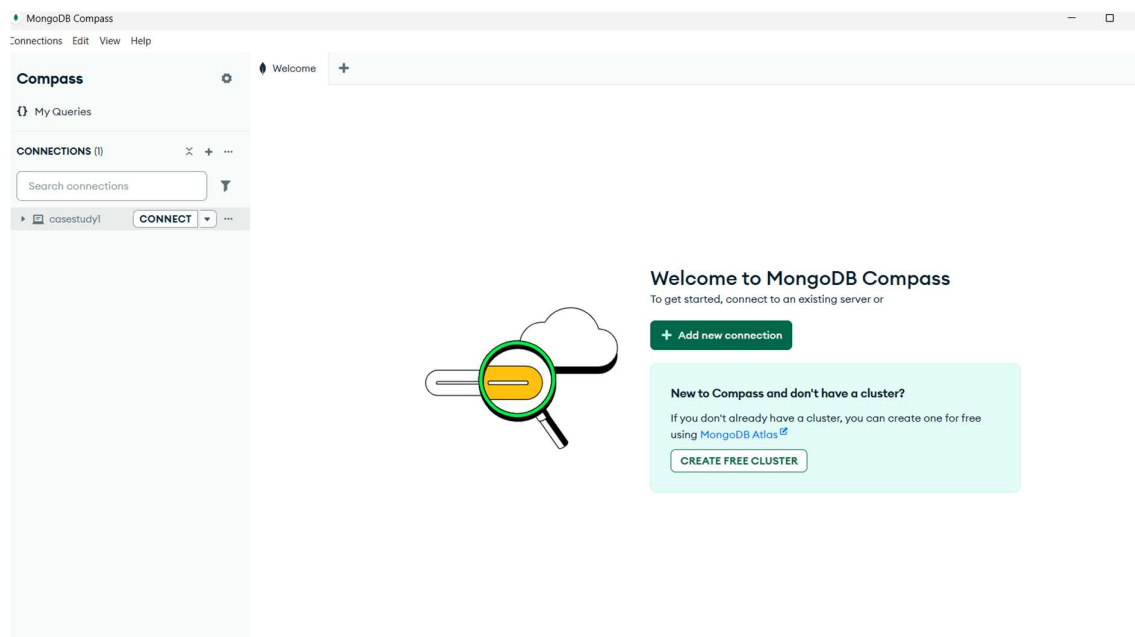
Description:

MongoDB Compass is a GUI tool that allows you to interact with MongoDB databases visually.

Steps:

1. Open the **MongoDB Compass** application from your system.
2. Click **Connect** to connect to your local MongoDB server.

Screenshot

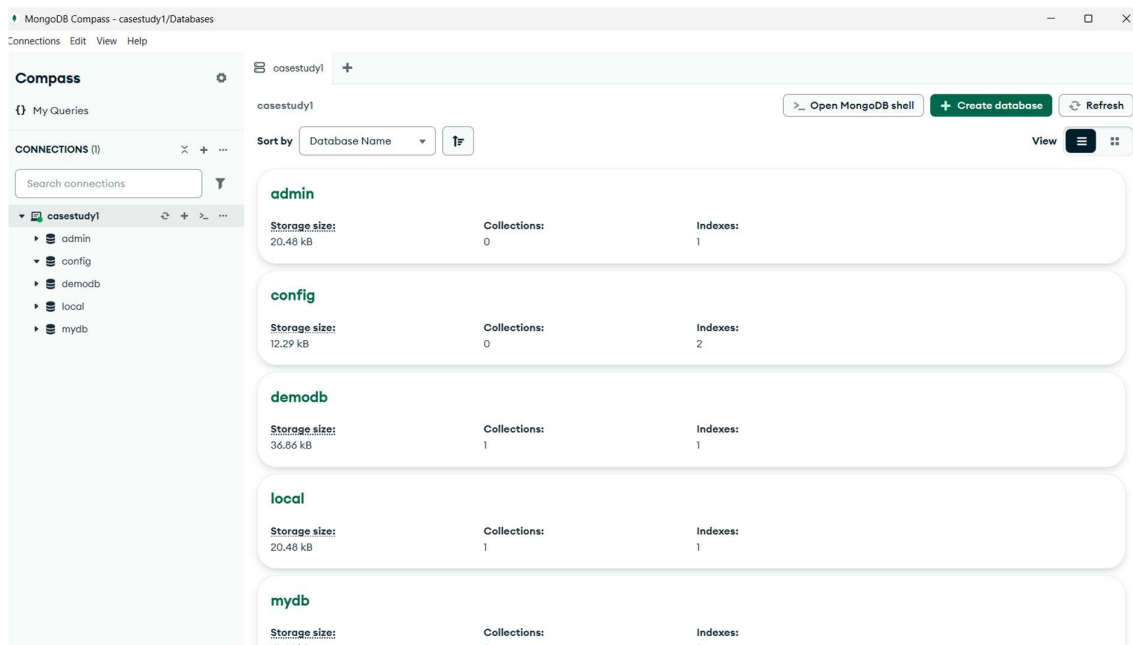


Step 2: Create Database and First Collection (users)

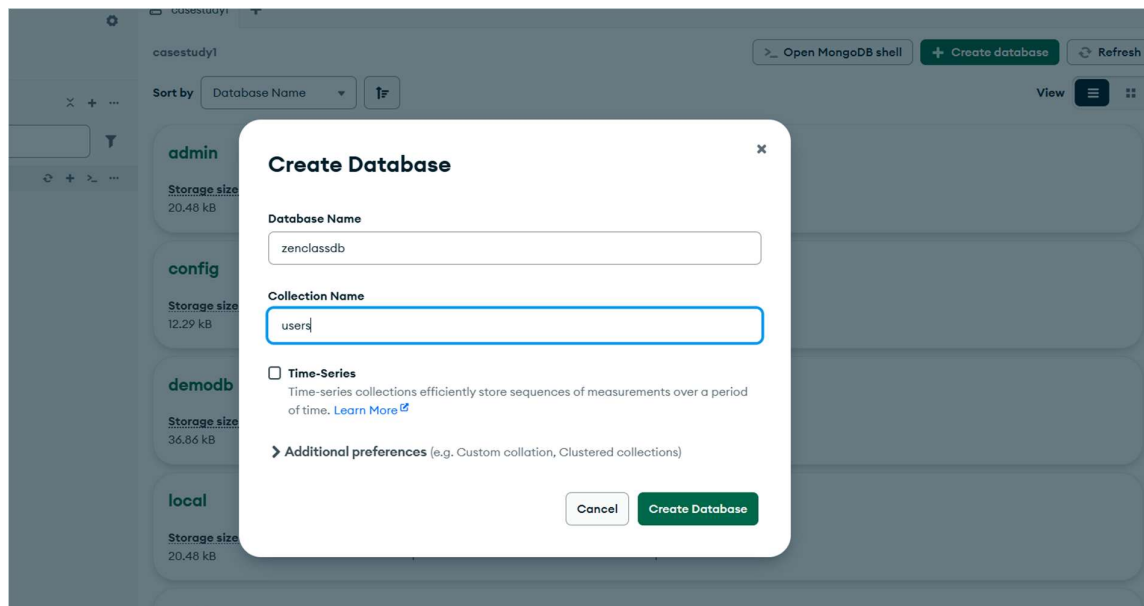
Steps:

1. Click on "Create Database".
2. Enter:
 - Database Name: zenclass
 - Collection Name: users
3. Click Create Database.

Screenshot 1



Screenshot 2



Step 3: Collections Creation, Import, and Verification

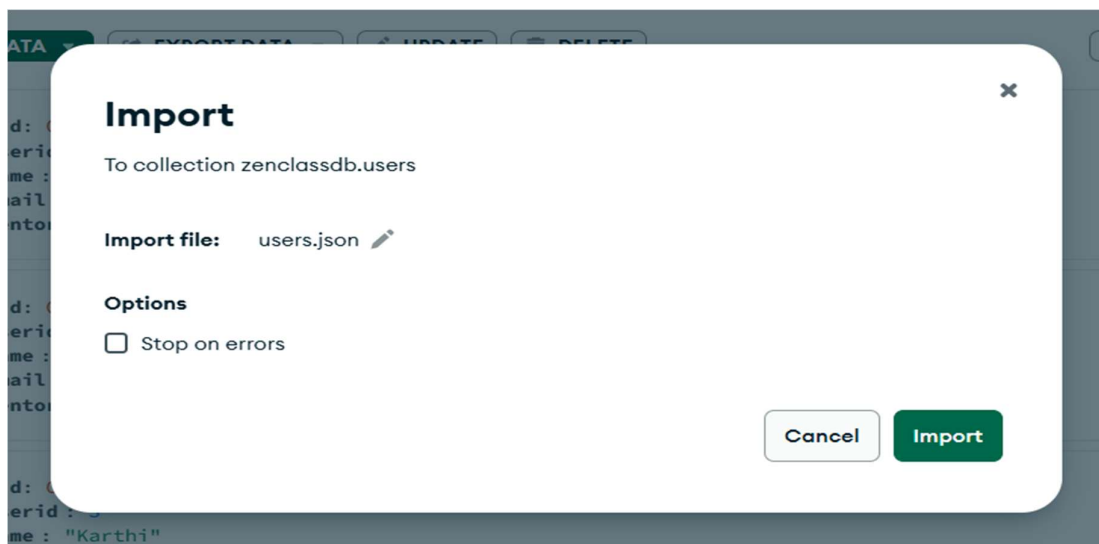
This step covers creating 8 collections, importing JSON files, and verifying documents using MongoDB Shell & Compass.

◆ 3.1: Collection – users

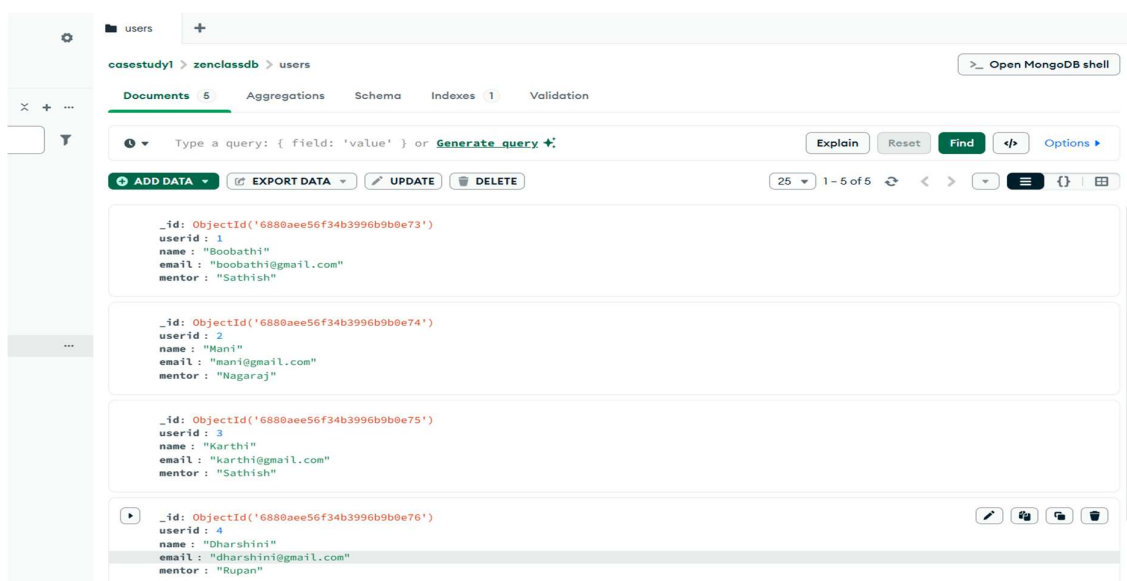
3.1.1: Create users Collection

- Click the users collection under zenclass.
- Click Add Data > Import File.
- Choose the file users.json.
- File Type: JSON
- Click Import.

📸 Screenshot 1



📸 Screenshot 2



3.2.1: Verify Imported Data in Mongo Shell

Steps:

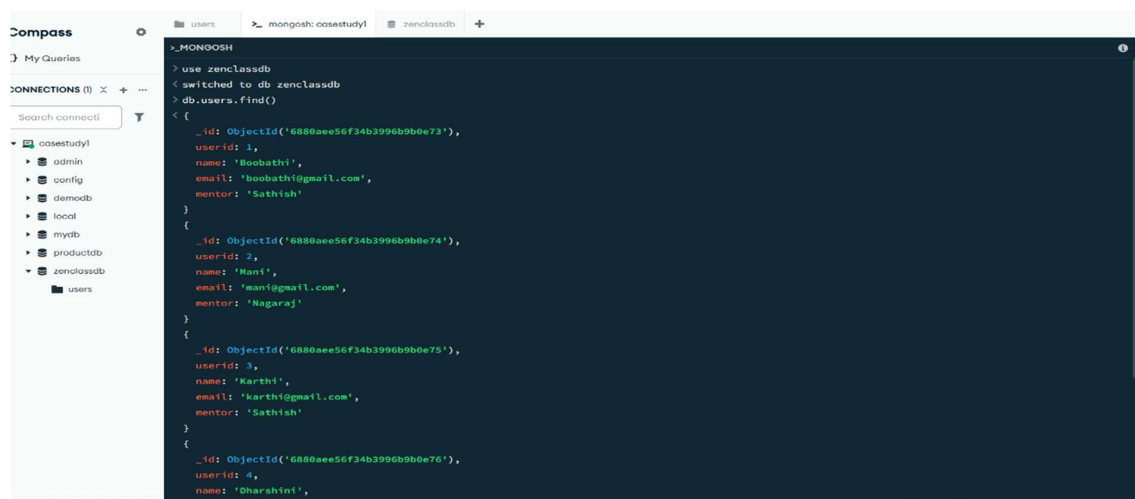
1. Open **Mongo Shell** from Compass.
2. Type the below command:

use zenclass

3. Display documents:

db.users.find()

Screenshot

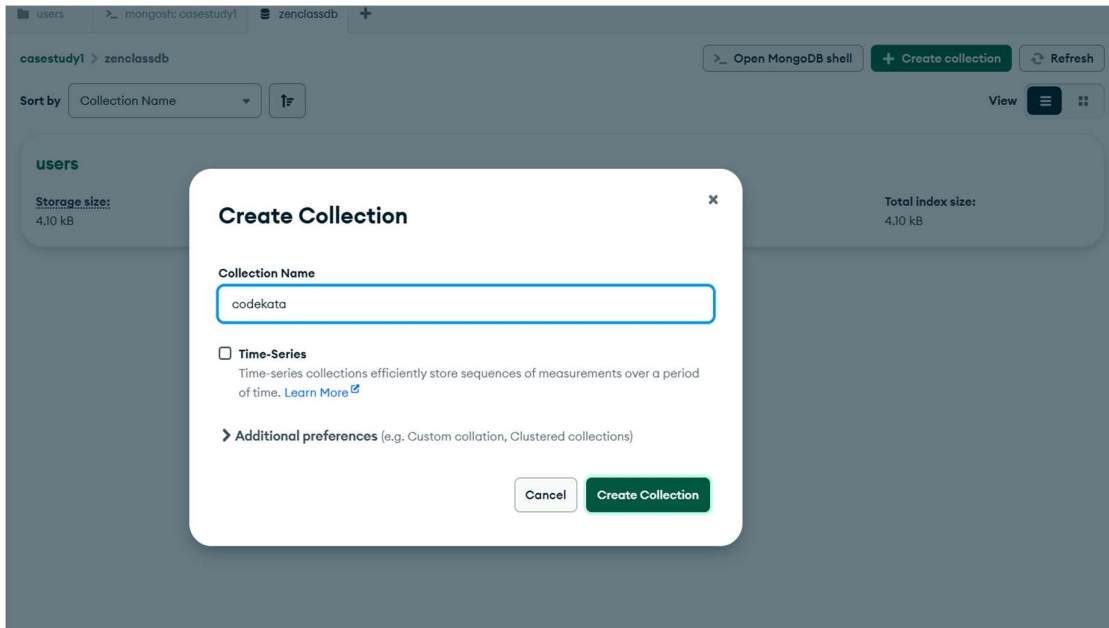


◆ 3.2: Collection – codekata

3.2.1: Create the codekata Collection

- In zenclass DB, click **“Create Collection”**
- Enter Name: codekata and click **Create**

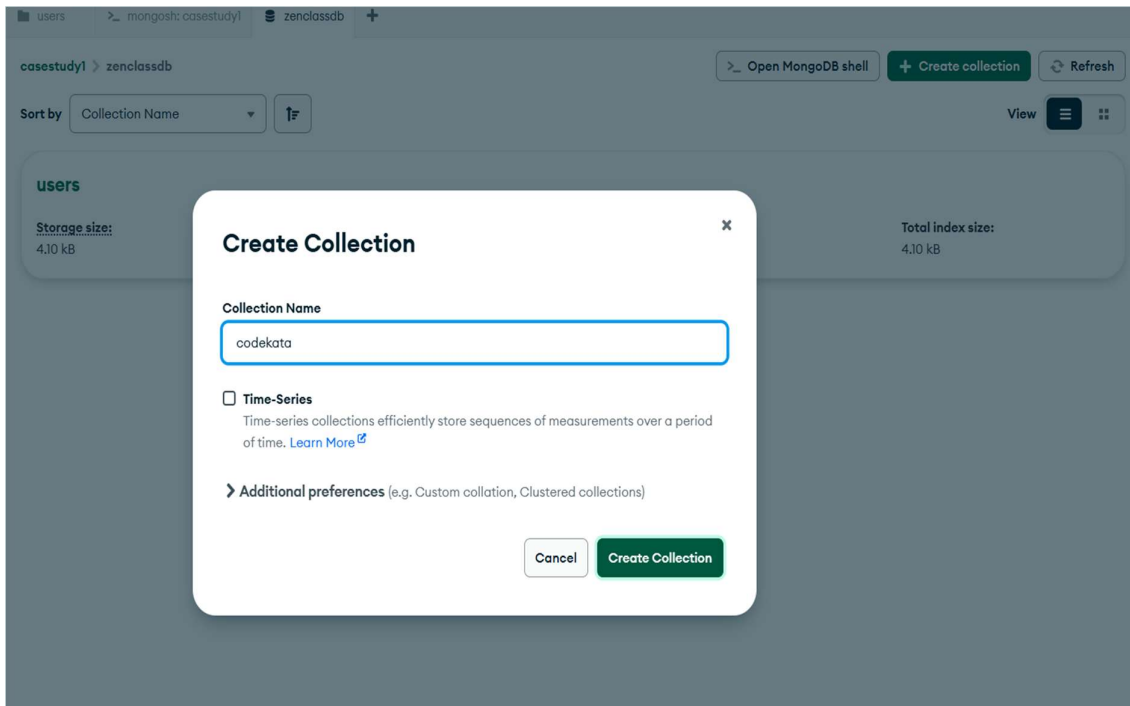
Screenshot



3.2.2: Import codekata.json

- Click codekata collection → **Add Data > Import File**
- Select codekata.json → File Type: JSON → Click **Import**

Screenshot

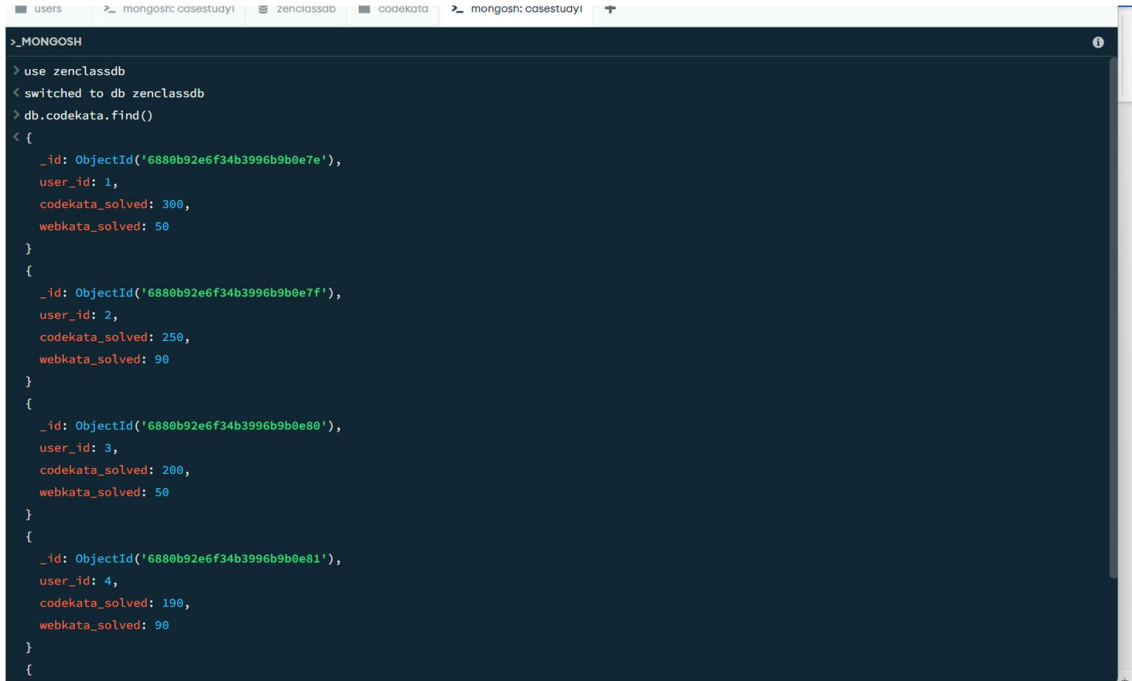


3.2.3: Verify Data

Open mongoshell in compass and type:

```
db.codekata.find()
```

Screenshot



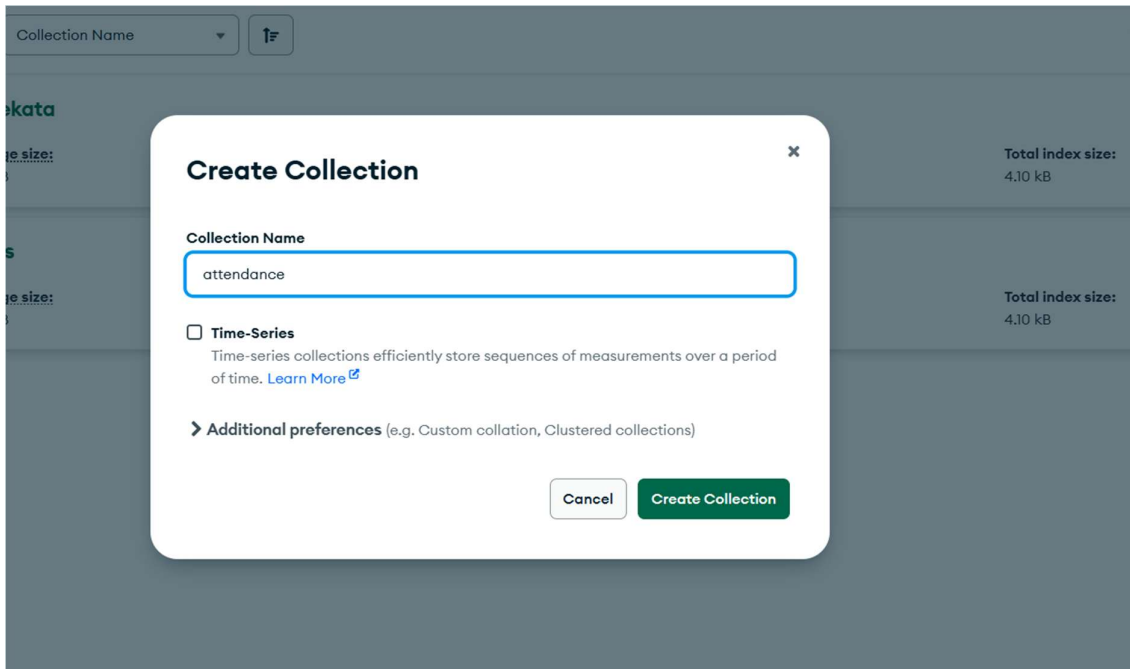
```
> use zenclassdb
< switched to db zenclassdb
> db.codekata.find()
< {
  _id: ObjectId('6880b92e6f34b3996b9b0e7e'),
  user_id: 1,
  codekata_solved: 300,
  webkata_solved: 50
}
{
  _id: ObjectId('6880b92e6f34b3996b9b0e7f'),
  user_id: 2,
  codekata_solved: 250,
  webkata_solved: 90
}
{
  _id: ObjectId('6880b92e6f34b3996b9b0e80'),
  user_id: 3,
  codekata_solved: 200,
  webkata_solved: 50
}
{
  _id: ObjectId('6880b92e6f34b3996b9b0e81'),
  user_id: 4,
  codekata_solved: 190,
  webkata_solved: 90
}
{
```

◆ 3.3: Collection – attendance

3.3.1: Create the attendance Collection

- Inside zenclass, click “**Create Collection**”
- Name it: attendance → Click **Create**

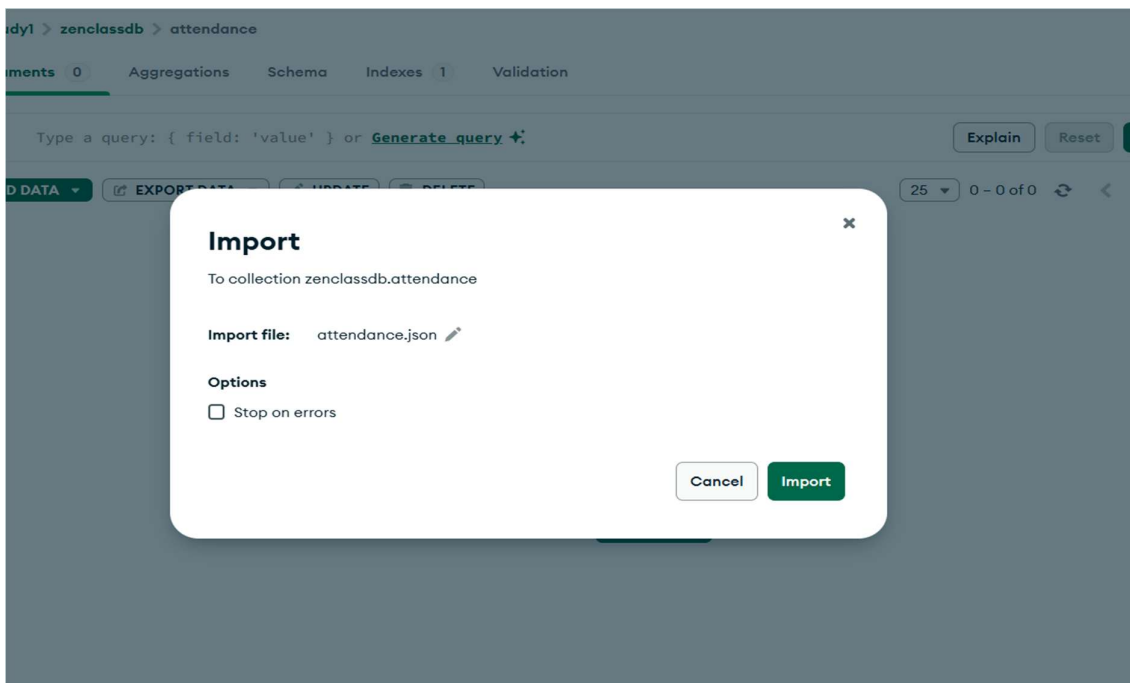
Screenshot



3.3.2: Import attendance.json

- Open the collection → Click **Add Data > Import File**
- Select attendance.json and Import.

Screenshot

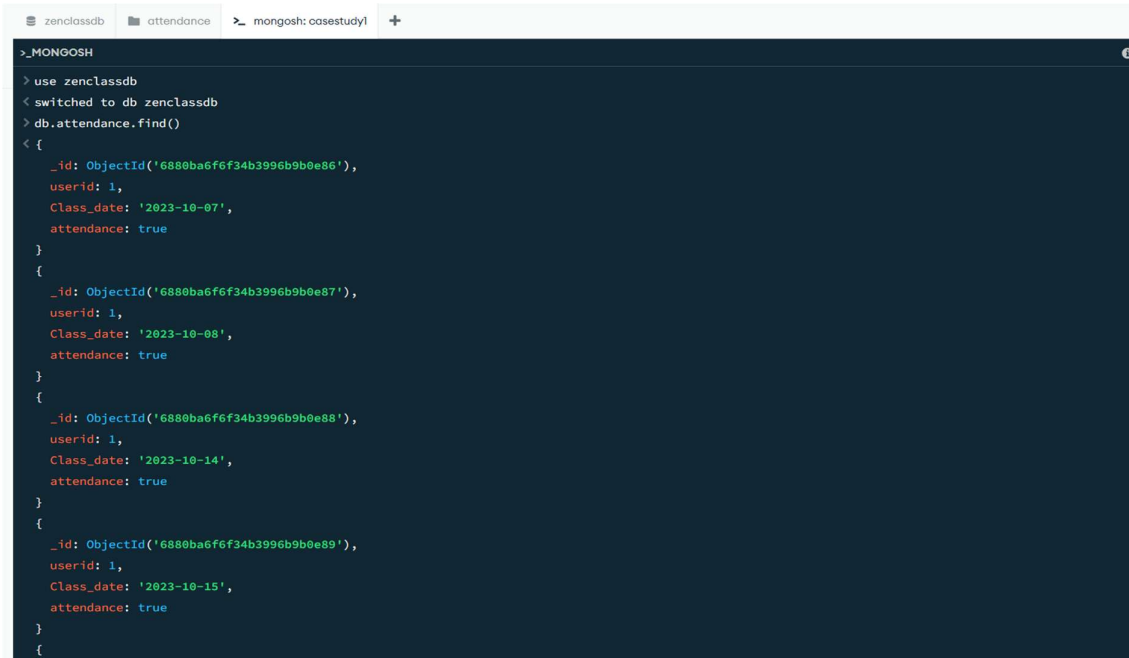


3.3.3: Verify Data

Click Open MongoDB shell in compass and type:

```
db.attendance.find()
```

Screenshot



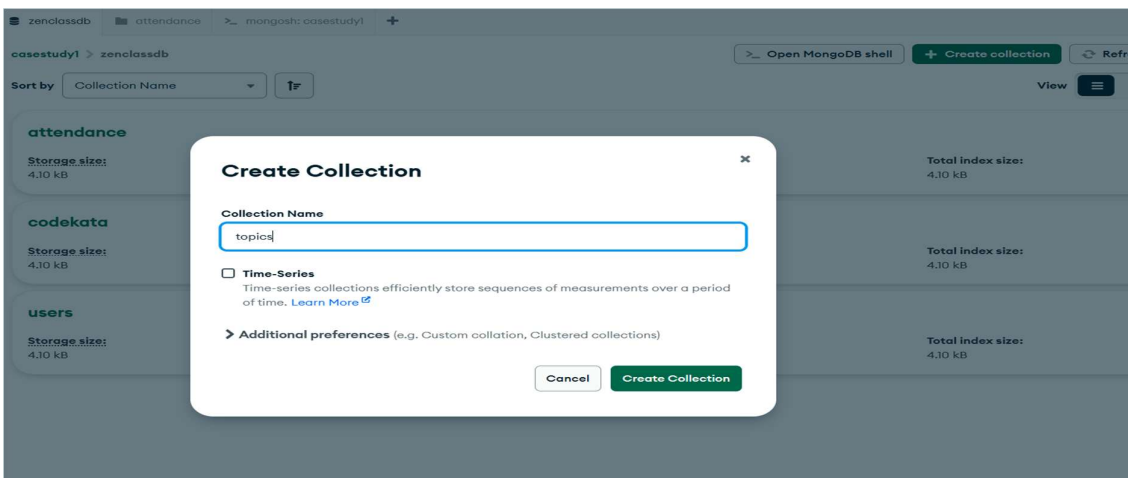
```
>_MONGOSH
> use zenclassdb
< switched to db zenclassdb
> db.attendance.find()
< {
  _id: ObjectId('6880ba6f6f34b3996b9b0e86'),
  userid: 1,
  Class_date: '2023-10-07',
  attendance: true
}
{
  _id: ObjectId('6880ba6f6f34b3996b9b0e87'),
  userid: 1,
  Class_date: '2023-10-08',
  attendance: true
}
{
  _id: ObjectId('6880ba6f6f34b3996b9b0e88'),
  userid: 1,
  Class_date: '2023-10-14',
  attendance: true
}
{
  _id: ObjectId('6880ba6f6f34b3996b9b0e89'),
  userid: 1,
  Class_date: '2023-10-15',
  attendance: true
}
{
```

◆ 3.4: Collection – topics

3.4.1: Create the topics Collection

- Go to zenclass → Click **Create Collection** → Name: topics

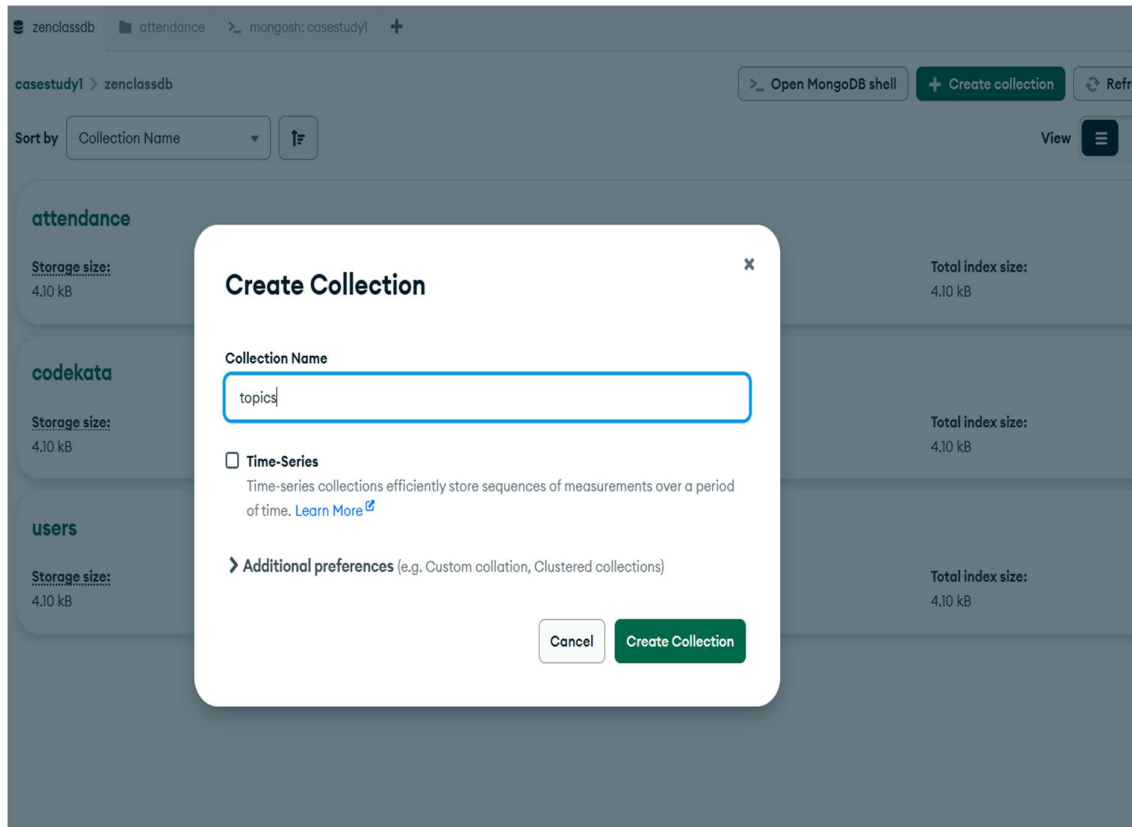
Screenshot



3.4.2: Import topics.json

- Open topics → Click **Add Data > Import File**
- Upload topics.json and import.

Screenshot

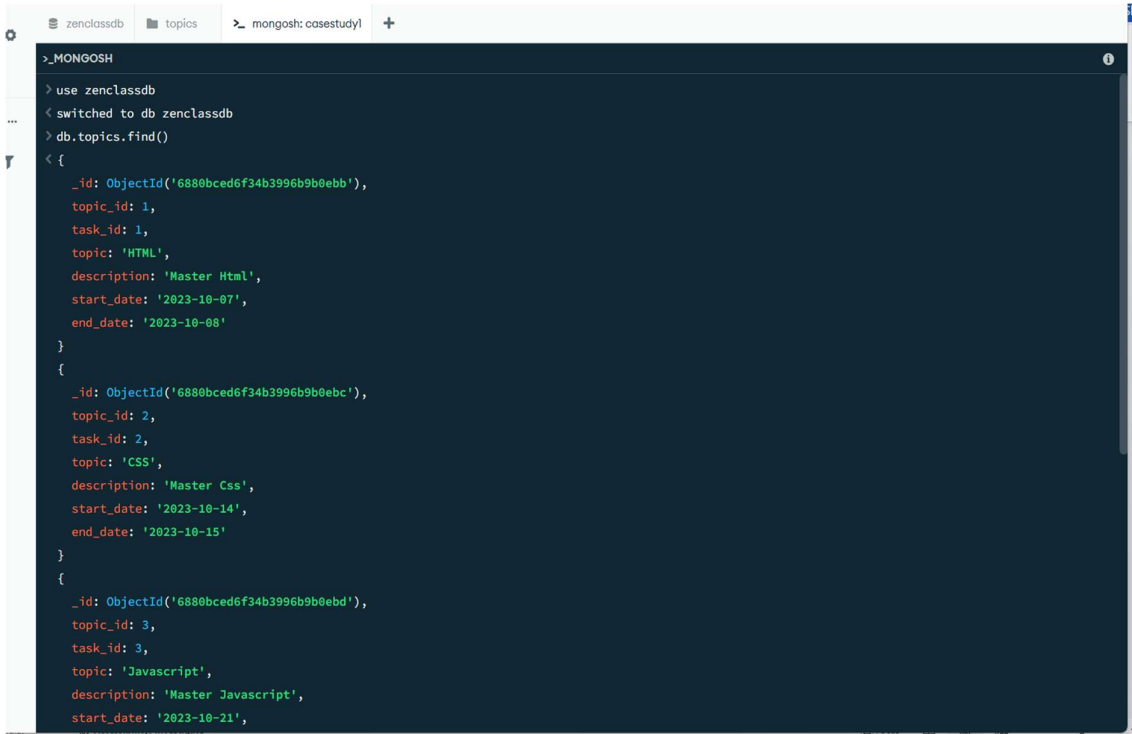


3.4.3: Verify Topics Data

Click Open MongoDB shell in compass and type:

```
db.topics.find()
```

Screenshot



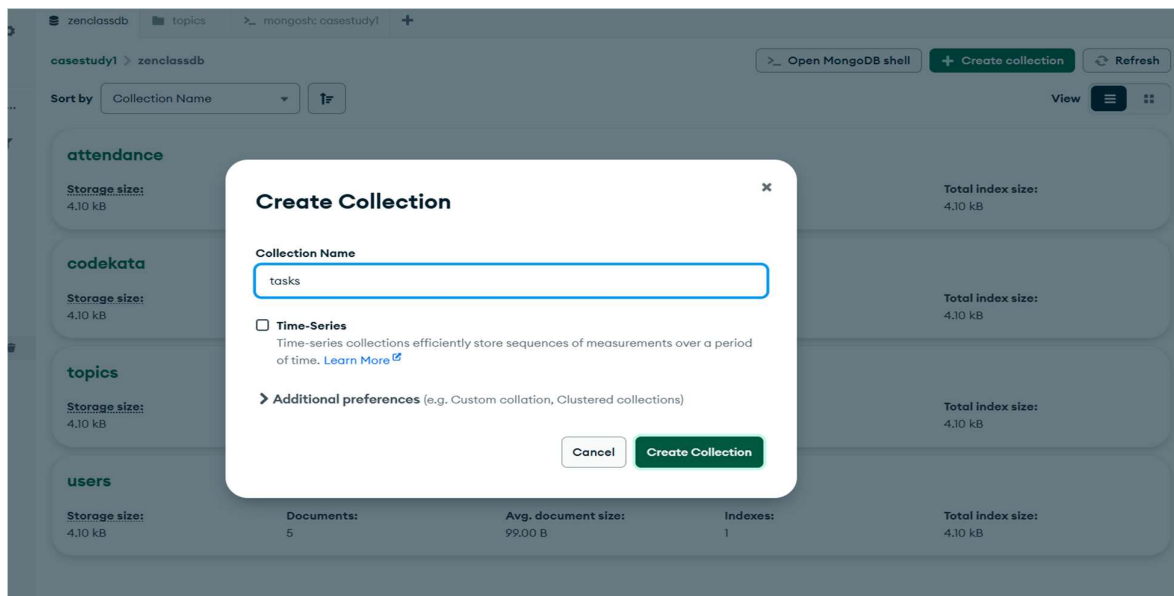
```
> use zenclassdb
switched to db zenclassdb
> db.topics.find()
{
  "_id": ObjectId("6880bcd6f34b3996b9b0ebb"),
  "topic_id": 1,
  "task_id": 1,
  "topic": "HTML",
  "description": "Master Html",
  "start_date": "2023-10-07",
  "end_date": "2023-10-08"
}
{
  "_id": ObjectId("6880bcd6f34b3996b9b0ebc"),
  "topic_id": 2,
  "task_id": 2,
  "topic": "CSS",
  "description": "Master Css",
  "start_date": "2023-10-14",
  "end_date": "2023-10-15"
}
{
  "_id": ObjectId("6880bcd6f34b3996b9b0ebd"),
  "topic_id": 3,
  "task_id": 3,
  "topic": "Javascript",
  "description": "Master Javascript",
  "start_date": "2023-10-21",
  "end_date": "2023-10-22"
}
```

◆ 3.5: Collection – tasks

3.5.1: Create the tasks Collection

- In zenclass, click **Create Collection** → Name it: tasks

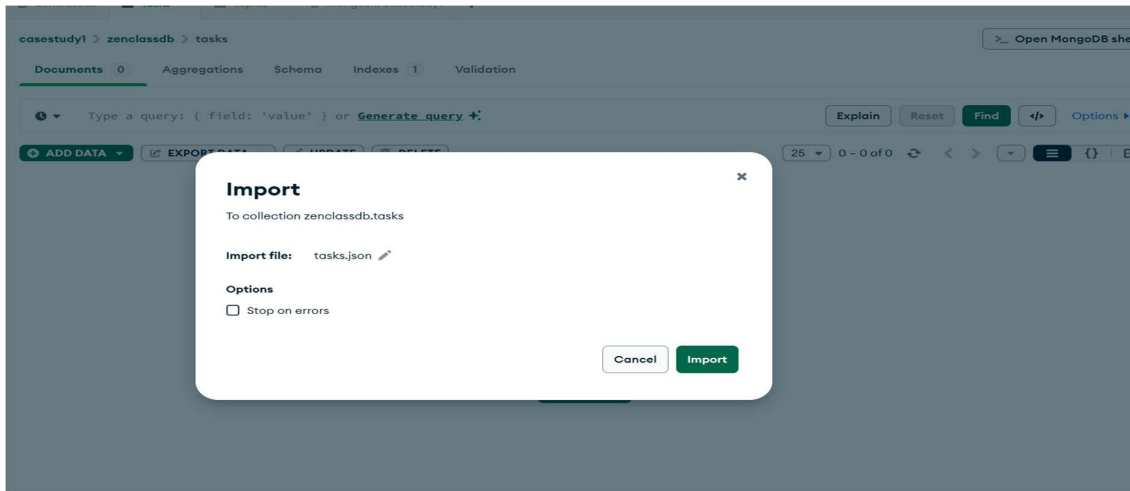
Screenshot



3.5.2: Import tasks.json

- Open the tasks collection → Click **Add Data**
- Choose tasks.json file and Import.

Screenshot

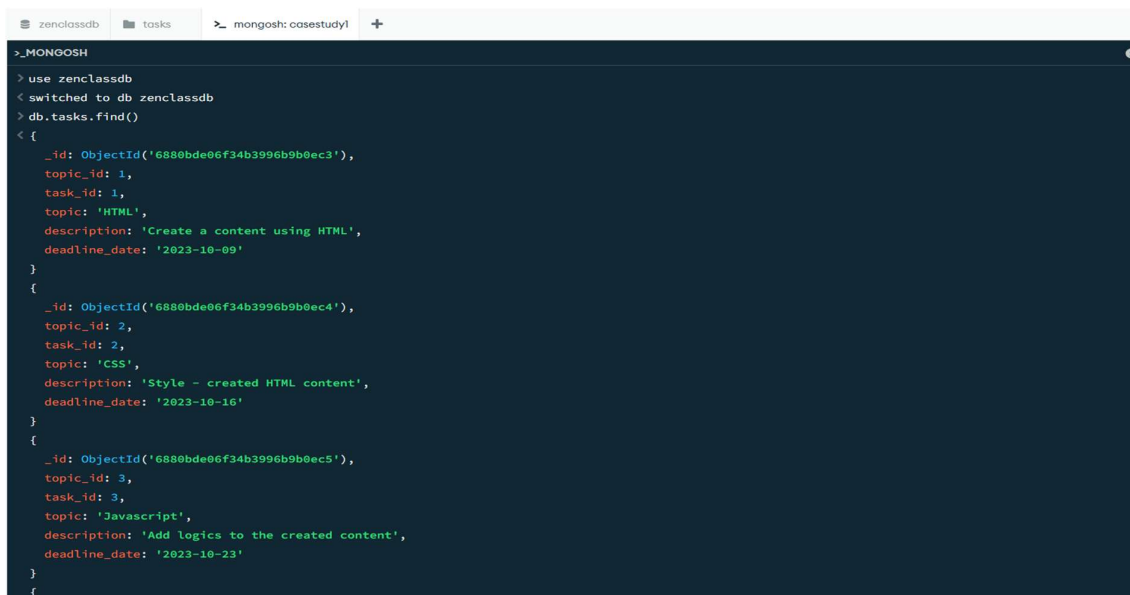


3.5.3: Verify Task Data

Click Open MongoDB shell in compass and type:

```
db.tasks.find()
```

Screenshot

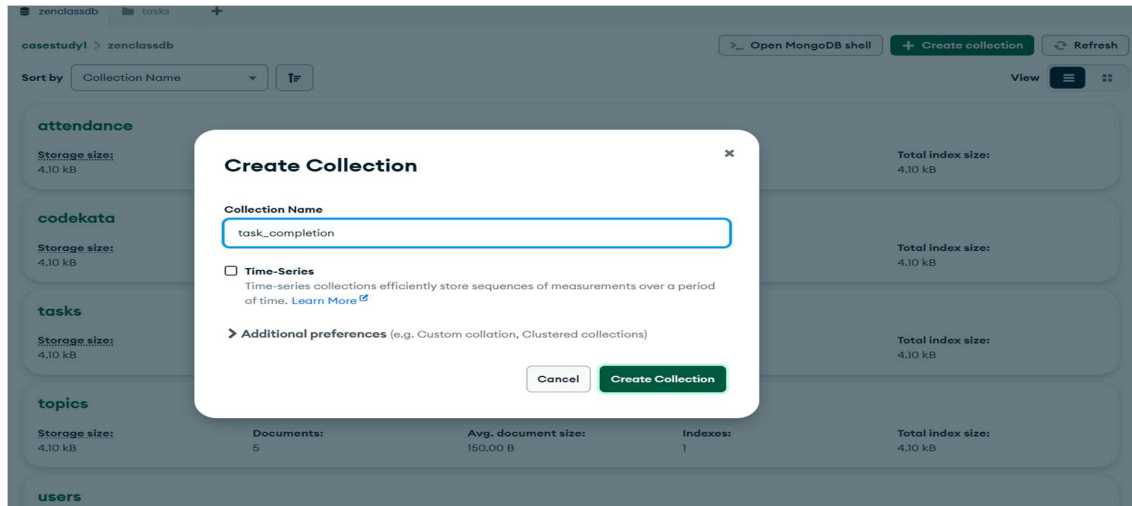


◆ 3.6: Collection – task_completion

3.6.1: Create the task_completion Collection

- Create new collection named task_completion under zenclass.

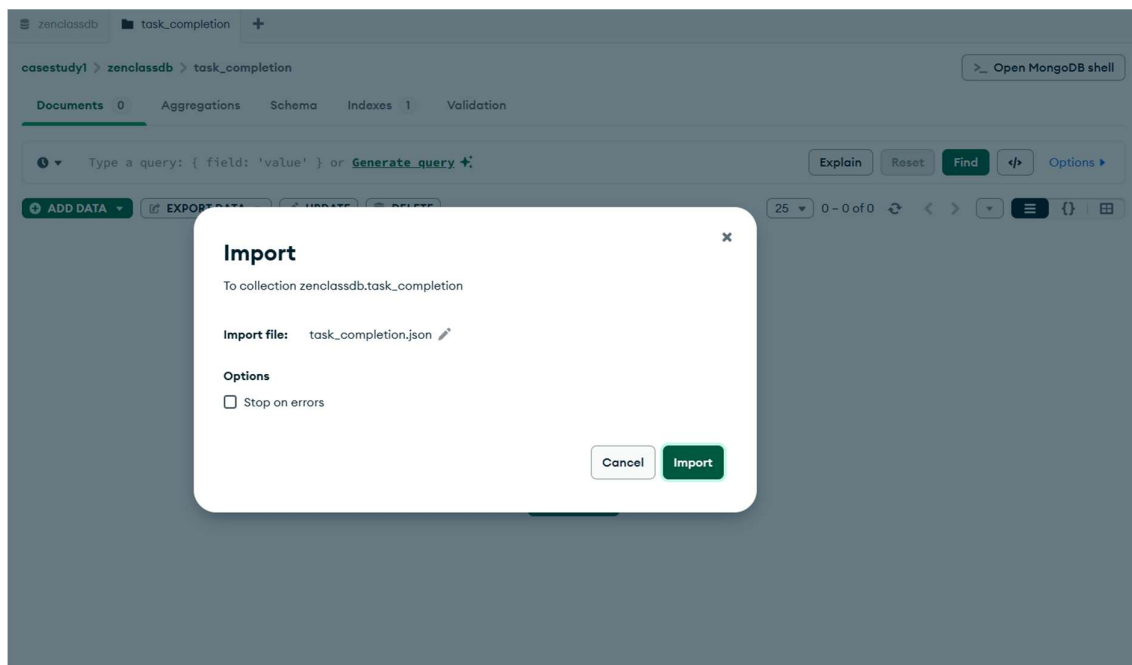
📸 Screenshot



3.6.2: Import task_completion.json

- Open collection → Click **Add Data > Import File**
- Upload task_completion.json

📸 Screenshot

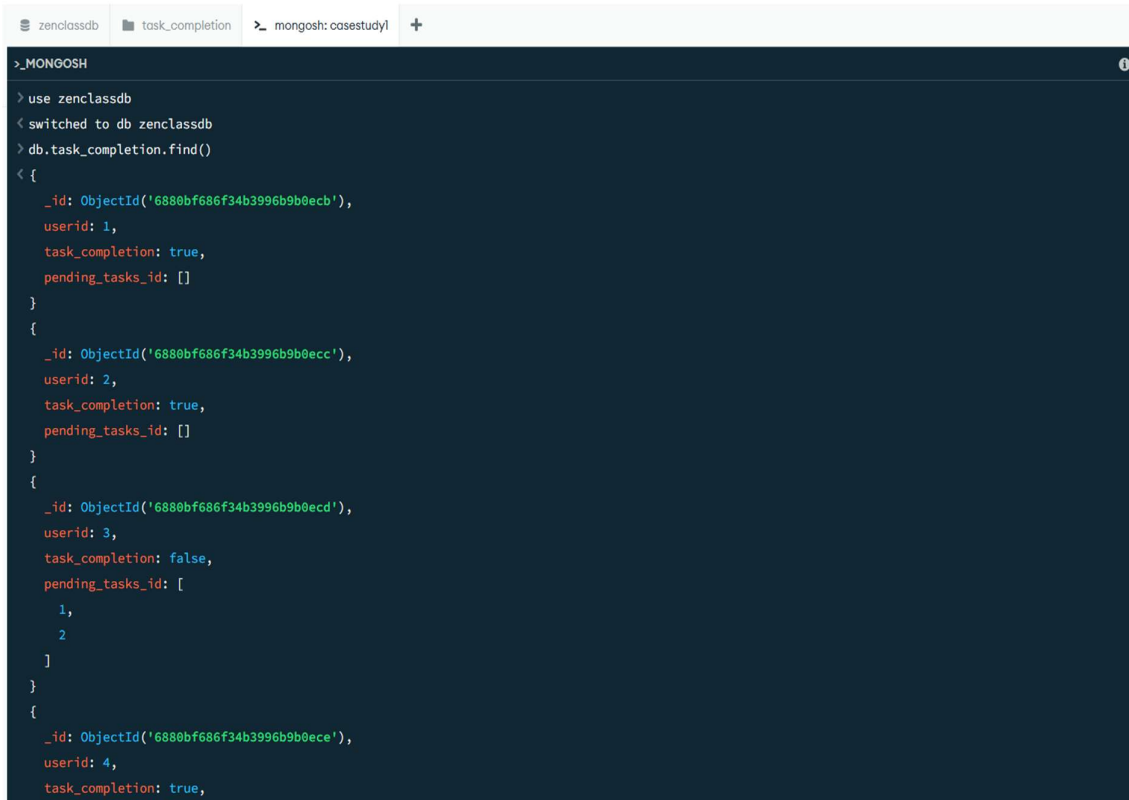


3.6.3: Verify Task Completion Data

Click Open MongoDB shell in compass and type:

```
db.task_completion.find()
```

Screenshot



The screenshot shows the MongoDB Compass interface with the 'task_completion' collection selected. The MongoDB shell output displays the following JSON documents:

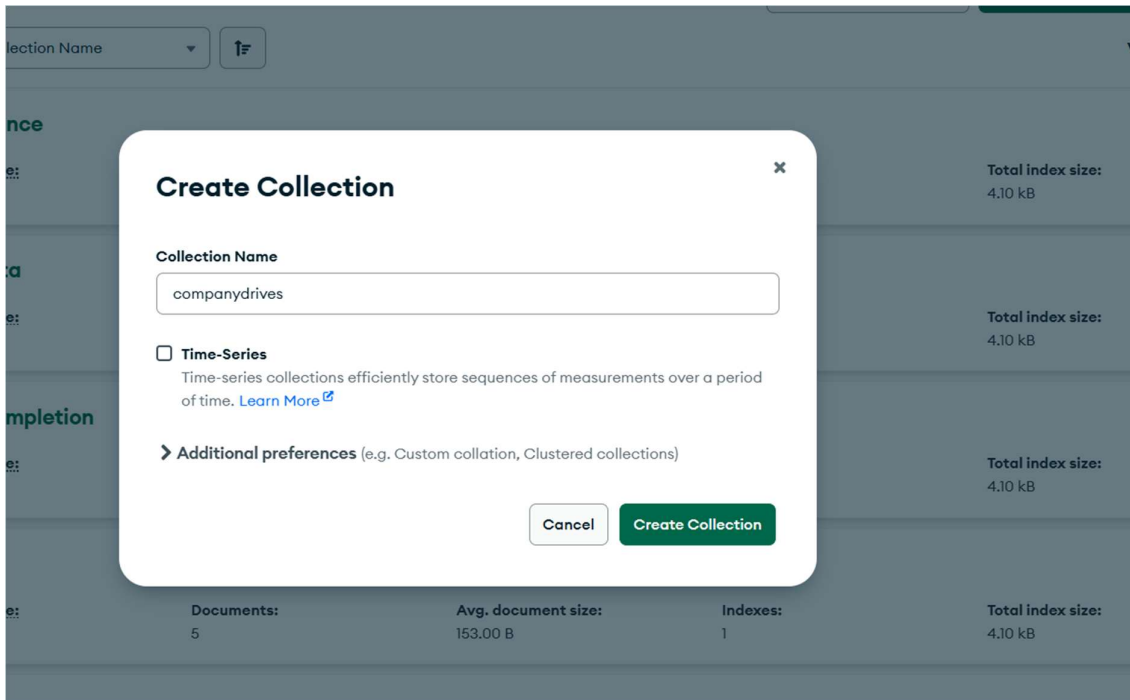
```
> use zenclassdb
< switched to db zenclassdb
> db.task_completion.find()
< {
  _id: ObjectId('6880bf686f34b3996b9b0ecb'),
  userid: 1,
  task_completion: true,
  pending_tasks_id: []
}
{
  _id: ObjectId('6880bf686f34b3996b9b0ecc'),
  userid: 2,
  task_completion: true,
  pending_tasks_id: []
}
{
  _id: ObjectId('6880bf686f34b3996b9b0ecd'),
  userid: 3,
  task_completion: false,
  pending_tasks_id: [
    1,
    2
  ]
}
{
  _id: ObjectId('6880bf686f34b3996b9b0ece'),
  userid: 4,
  task_completion: true,
```

◆ 3.7: Collection – companydrives

3.7.1: Create companydrives Collection

- Create collection named companydrives under zenclass.

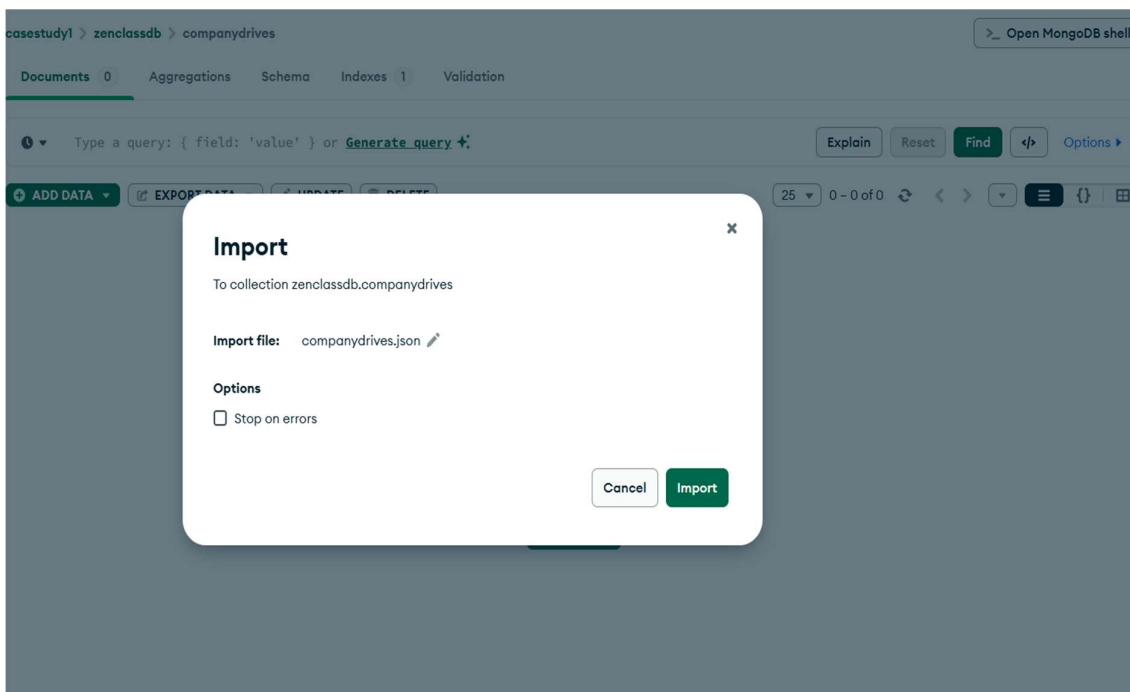
Screenshot



3.7.2: Import companydrives.json

- Click on collection → Import File → Select companydrives.json

Screenshot

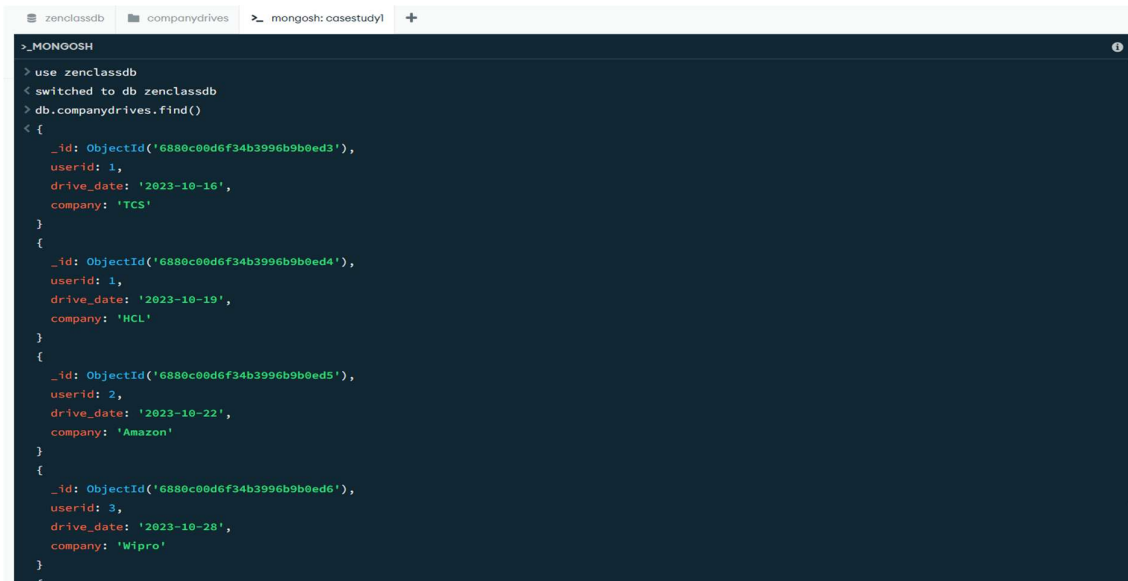


3.7.3: Verify Company Drives

Click Open MongoDB shell in compass and type:

```
db.companydrives.find()
```

Screenshot



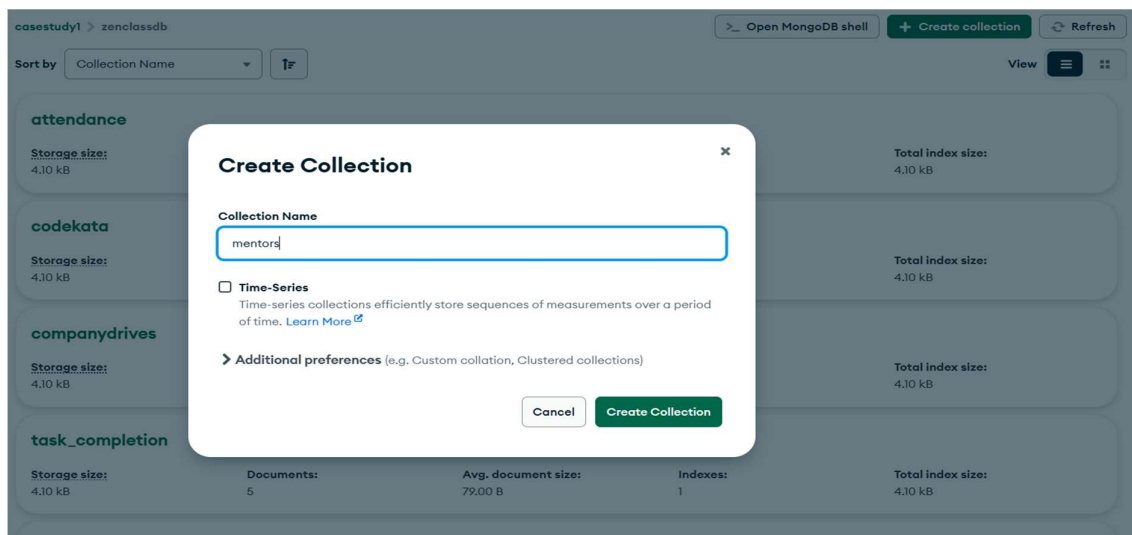
```
> use zenclassdb
< switched to db zenclassdb
> db.companydrives.find()
{
  "_id": ObjectId("6880c0d6f34b3996b9b0ed3"),
  "userid": 1,
  "drive_date": "2023-10-16",
  "company": "TCS"
}
{
  "_id": ObjectId("6880c0d6f34b3996b9b0ed4"),
  "userid": 1,
  "drive_date": "2023-10-19",
  "company": "HCL"
}
{
  "_id": ObjectId("6880c0d6f34b3996b9b0ed5"),
  "userid": 2,
  "drive_date": "2023-10-22",
  "company": "Amazon"
}
{
  "_id": ObjectId("6880c0d6f34b3996b9b0ed6"),
  "userid": 3,
  "drive_date": "2023-10-28",
  "company": "Wipro"
}
```

◆ 3.8: Collection – mentors

3.8.1: Create mentors Collection

- Final collection → Create mentors

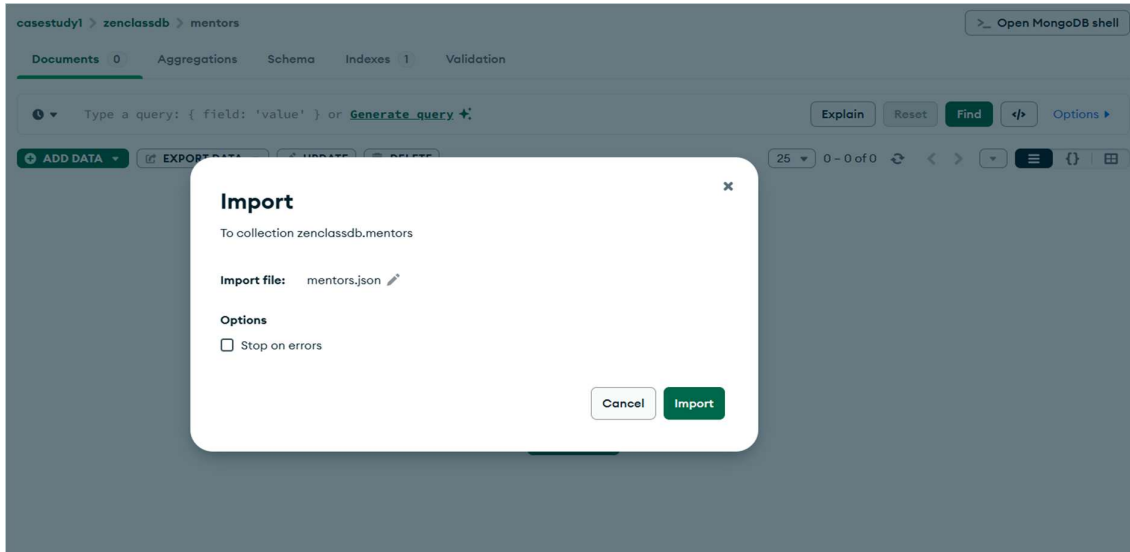
Screenshot



3.8.2: Import mentors.json

- Click on collection → Add Data → Import File → Select mentors.json

Screenshot

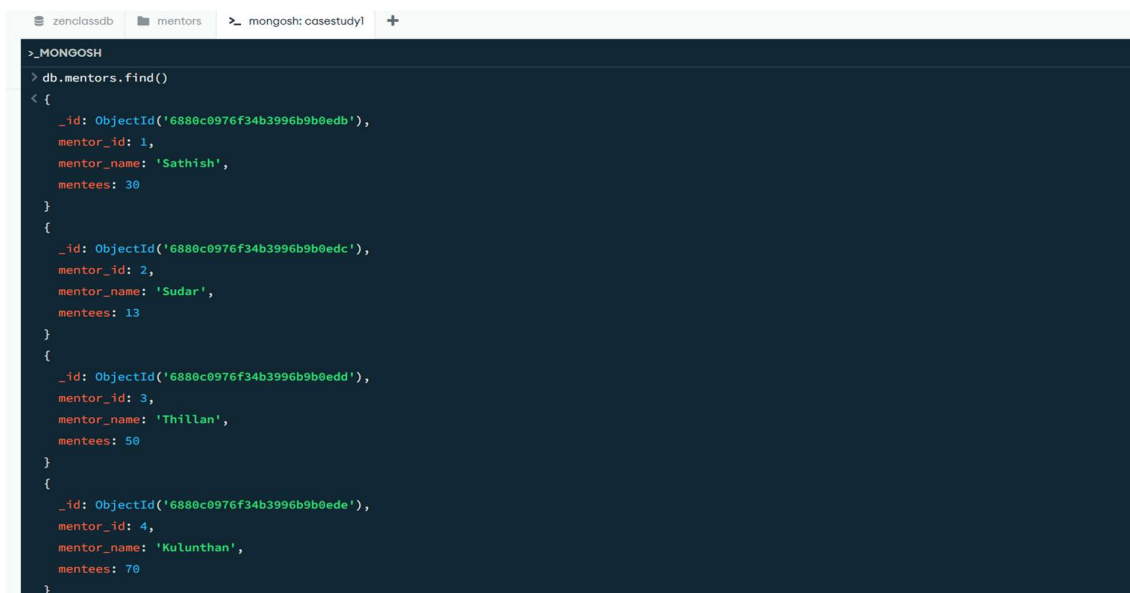


3.8.3: Verify Mentor Records

Click Open MongoDB shell in compass and type:

```
db.mentors.find()
```

Screenshot



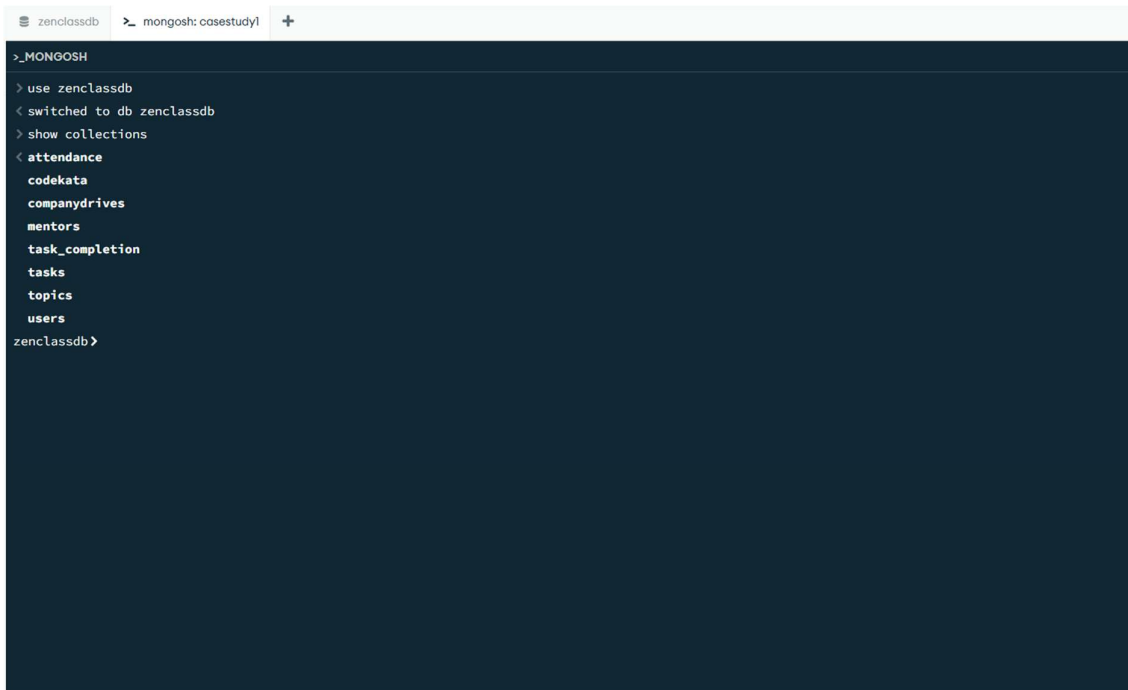
✅ Step 4: Verification of All Collections

After importing all collections, confirm their presence by running:

use zenclass

show collections

🖼️ Screenshot



```
zenclassdb > mongosh: casestudy1 +
_MONGOSH
> use zenclassdb
< switched to db zenclassdb
> show collections
< attendance
  codekata
  companydrives
  mentors
  task_completion
  tasks
  topics
  users
zenclassdb>
```

✅ Step 5: Execute the Queries

1. Find all the topics and tasks which were taught in October 2023

🔍 Query for topics:

```
db.topics.find({
  start_date: {
    $gte: ("2023-10-01"),
    $lte: ("2023-10-31") }});
```

Screenshot

```
mongoosha
  topics
    mongoosha case study
  tasks
    +

> db.topics.find({
  start_date: {
    $gte: "2023-10-01",
    $lte: "2023-10-31"
  }
});
[
  {
    _id: ObjectId('6881b6403b536f41e6ca09c3'),
    topic_id: 1,
    task_id: 1,
    topic: 'HTML',
    description: 'Master Html',
    start_date: '2023-10-07',
    end_date: '2023-10-08'
  },
  {
    _id: ObjectId('6881b6403b536f41e6ca09c4'),
    topic_id: 2,
    task_id: 2,
    topic: 'CSS',
    description: 'Master Css',
    start_date: '2023-10-14',
    end_date: '2023-10-15'
  },
  {
    _id: ObjectId('6881b6403b536f41e6ca09c5'),
    topic_id: 3,
    task_id: 3,
    topic: 'Javascript',
    description: 'Master Javascript',
    start_date: '2023-10-21',
    end_date: '2023-10-22'
  },
  {
    _id: ObjectId('6881b6403b536f41e6ca09c6'),
    topic_id: 4,
    task_id: 4,
    topic: 'React',
    description: 'Master React',
    start_date: '2023-10-28',
    end_date: '2023-10-29'
  }
]
```

Query for tasks:

db.tasks.find({deadline_date: { \$gte: ("2023-10-01"),\$lte: (2023-10-31)" }});

Screenshot 2

```
db.tasks.find({
  deadline_date: {
    $gte: "2023-10-01",
    $lte: "2023-10-31"
  }
});
[
  {
    _id: ObjectId('6881b62f3b536f41e6ca09bd'),
    topic_id: 1,
    task_id: 1,
    topic: 'HTML',
    description: 'Create a content using HTML',
    deadline_date: '2023-10-09'
  },
  {
    _id: ObjectId('6881b62f3b536f41e6ca09be'),
    topic_id: 2,
    task_id: 2,
    topic: 'CSS',
    description: 'Style - created HTML content',
    deadline_date: '2023-10-16'
  },
  {
    _id: ObjectId('6881b62f3b536f41e6ca09bf'),
    topic_id: 3,
    task_id: 3,
    topic: 'Javascript',
    description: 'Add logics to the created content',
    deadline_date: '2023-10-23'
  },
  {
    _id: ObjectId('6881b62f3b536f41e6ca09c0'),
    topic_id: 4,
    task_id: 4,
    topic: 'React',
    description: 'Create same website (created by using of html,css & js) using react',
    deadline_date: '2023-10-30'
  }
]
```

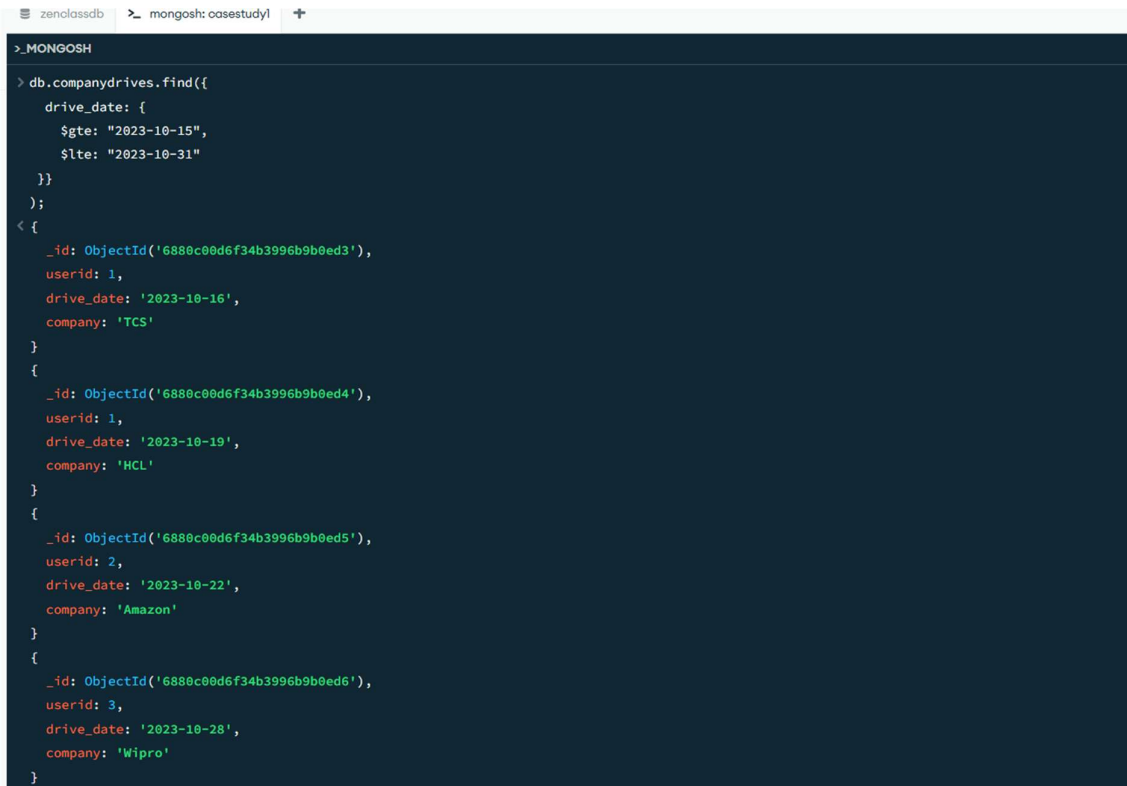
Explanation:

These queries check start_date for topics and deadline_date for tasks that fall within October 2023.

2. Find all the company drives between 15-Oct-2023 and 31-Oct-2023

```
db.companydrives.find({
  drive_date: {
    $gte: ("2023-10-15"),
    $lte: ("2023-10-31") }});
```

Screenshot



The screenshot shows a MongoDB terminal window with the following content:

```
>_MONGOSH
> db.companydrives.find({
  drive_date: {
    $gte: "2023-10-15",
    $lte: "2023-10-31"
  }}
);
< {
  _id: ObjectId('6880c00d6f34b3996b9b0ed3'),
  userid: 1,
  drive_date: '2023-10-16',
  company: 'TCS'
}
{
  _id: ObjectId('6880c00d6f34b3996b9b0ed4'),
  userid: 1,
  drive_date: '2023-10-19',
  company: 'HCL'
}
{
  _id: ObjectId('6880c00d6f34b3996b9b0ed5'),
  userid: 2,
  drive_date: '2023-10-22',
  company: 'Amazon'
}
{
  _id: ObjectId('6880c00d6f34b3996b9b0ed6'),
  userid: 3,
  drive_date: '2023-10-28',
  company: 'Wipro'
}
```

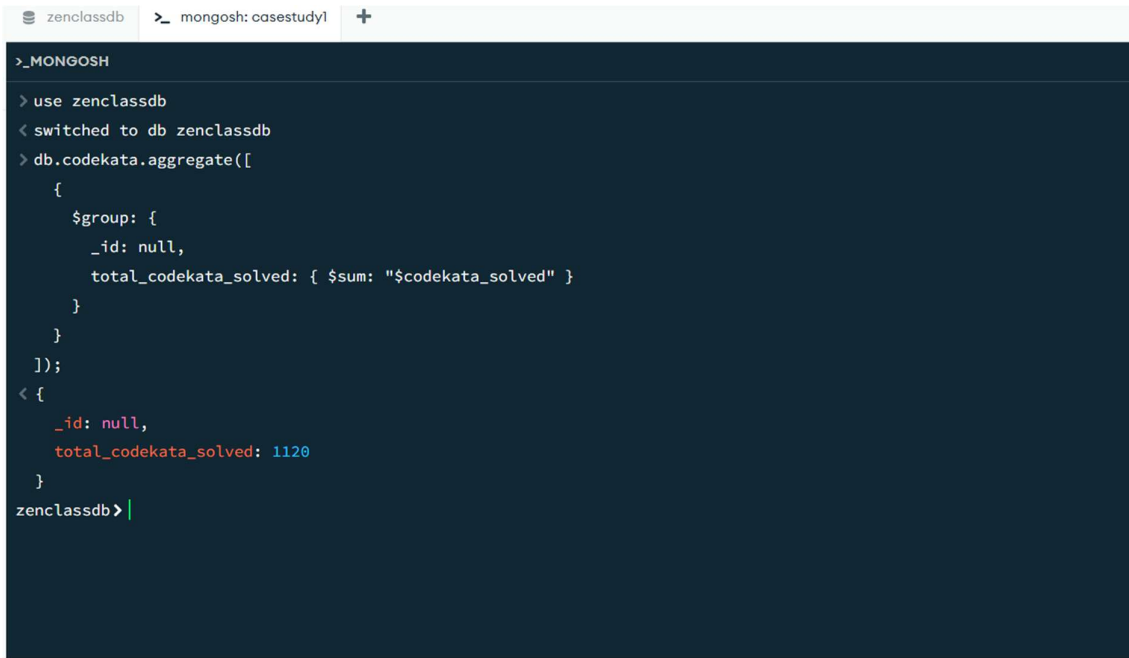
Explanation:

Retrives Matches companies like TCS, HCL, Amazon that conducted drives in the specified range.

✅ 3. Find the total problems solved by each user in codekata

```
db.codekata.aggregate({ $group: { _id: null,
    total_codekata_solved: { $sum: "$codekata_solved" }
  }
});
```

Screenshot



The screenshot shows a MongoDB terminal window with the following content:

```
>_MONGOSH
> use zenclassdb
< switched to db zenclassdb
> db.codekata.aggregate([
  {
    $group: {
      _id: null,
      total_codekata_solved: { $sum: "$codekata_solved" }
    }
  }
]);
< {
  _id: null,
  total_codekata_solved: 1120
}
zenclassdb>
```

Explanation:

This query uses the **\$group stage** in aggregation to **calculate the total number of problems solved in codekata_solved by all users combined**.

✅ 4. Find all mentors who have more than 15 mentees

```
db.mentors.find(
{
  mentees: { $gt: 15 }
}
);
```

Screenshot

```
zenclassdb | mongo: casestudy |
>_MONGOSH
> use zenclassdb
< switched to db zenclassdb
> db.mentors.find({
  mentees: { $gt: 15 }
});
{
  {
    _id: ObjectId('6880c0976f34b3996b9b0edd'),
    mentor_id: 1,
    mentor_name: 'Sathish',
    mentees: 30
  },
  {
    _id: ObjectId('6880c0976f34b3996b9b0edd'),
    mentor_id: 3,
    mentor_name: 'Thillan',
    mentees: 50
  },
  {
    _id: ObjectId('6880c0976f34b3996b9b0ede'),
    mentor_id: 4,
    mentor_name: 'Kulunthan',
    mentees: 78
  },
  {
    _id: ObjectId('6880c0976f34b3996b9b0eed'),
    mentor_id: 5,
    mentor_name: 'Nani',
    mentees: 16
  },
  {
    _id: ObjectId('6880c0976f34b3996b9b0ee2'),
    mentor_id: 8,
    mentor_name: 'Parthiban',
    mentees: 32
  },
  {
    _id: ObjectId('6880c0976f34b3996b9b0ee4'),
    mentor_id: 10,
    mentor_name: 'Vijay',
    mentees: 35
  }
}
```

Explanation:

This query filters mentors like Sathish, Parthiban, Vijay, etc. with large batches.

✓ 5. Find number of users who were absent and didn't submit tasks between 15-Oct and 31-Oct-2023

// Step 1: Get absentees between 15-31 Oct

```
zenclassdb | mongo: casestudy | +
>_MONGOSH
> use zenclassdb
< switched to db zenclassdb
> const absentees = db.attendance.find({
  Class_date: {
    $gte: "2023-10-15",
    $lte: "2023-10-31"
  },
  attendance: false
}).toArray();
zenclassdb>
```

// Step 2: Extract distinct user IDs

```
    }).toArray();  
> const absentUserIds = [...new Set(absentees.map(a => a.userid))];  
zenclassdb>
```

// Step 3: Find users who also didn't complete tasks

```
> const incompleteUsers = db.task_completion.find({  
  userid: { $in: absentUserIds },  
  task_completion: false  
}).toArray();  
zenclassdb>
```

// Step 4: Count the users

```
> incompleteUsers.length;  
< 2  
zenclassdb>
```

Final Screenshot

```
> _MONGOSH  
  
> use zenclassdb  
< switched to db zenclassdb  
  
> const absentees = db.attendance.find({  
  Class_date: {  
    $gte: "2023-10-15",  
    $lte: "2023-10-31"  
  },  
  attendance: false  
}).toArray();  
  
> const absentUserIds = [...new Set(absentees.map(a => a.userid))];  
  
> const incompleteUsers = db.task_completion.find({  
  userid: { $in: absentUserIds },  
  task_completion: false  
}).toArray();  
  
> incompleteUsers.length;  
< 2  
zenclassdb>
```

Explanation:

This query identifies users who were **absent** and also **did not complete their tasks** between **15th October 2023 and 31st October 2023**.

- **Step 1:** It queries the attendance collection to get all users who were **marked absent** (attendance: false) during the given date range.
- **Step 2:** It extracts the **unique** userids of those absent users.

- **Step 3:** It queries the task_completion collection to find how many of those users also have task_completion: false (i.e., they **did not submit their tasks**).
 - **Step 4:** Finally, it returns the **total number of such users**.
-

Conclusion:

In this case study, I:

- Explored MongoDB Compass GUI.
- Created a database and imported 8 collections using .json files.
- Verified documents using Mongo Shell.
- Wrote and executed 5 real-world queries involving date filtering, aggregation, joins (via logic), and condition checks.
- Learned hands-on MongoDB operations for real-world datasets.